

TP2 : Date, Sérialisation et Gestion des Exceptions

Partie 1 : Classe Date & Exceptions (60 minutes)

Exercice 1 — Classe Date (30 min)

Objectifs : validation, invariants, ordre, parsing/formatting.

Énoncé :

Créez `datex.py`. Implémentez une classe `Date` (calendrier grégorien) sans utiliser `datetime.date` pour la validation (vous pouvez l'utiliser pour les tests si besoin).

Contraintes fonctionnelles :

1. Constructeur : `**__init__**__(self, year:int, month:int, day:int)`
2. Valider : `year` ∈ [1, 9999], `month` ∈ [1,12], `day` valide selon le mois et les années bissextiles.
3. En cas d'erreur : lever `InvalidDateError`.

2. Méthodes & propriétés :

- a. `is_leap_year(self) -> bool`
- b. `__repr__` et `__str__` (formats `Date(2025,09,22)` et `2025-09-22`)
- c. Comparaisons : `__eq__`, `__lt__` (et déduire les autres via `functools.total_ordering`).
- d. `to_iso(self) -> str` (YYYY-MM-DD)
- e. `@classmethod from_iso(cls, s:str) -> "Date"` : parser YYYY-MM-DD ou lever `DateParseError`.

4. Méthodes utilitaires :

- f. `day_of_year(self) -> int`
- g. `add_days(self, n:int) -> "Date"` (retourne un nouvel objet, ne modifie pas `self`).

Exceptions à définir :

```
# errors.py
class DateError(Exception): ...
class InvalidDateError(DateError): ...
```

```
class DateParseError(DateError): ...
```

```
Code de départ (datex.py) :
```

```
from __future__ import annotations
from functools import total_ordering
from typing import Tuple
from errors import InvalidDateError, DateParseError
```

```
@total_ordering
```

```
class Date:
```

```
    """Date grégorienne minimale, immuable en pratique (méthodes
    renvoient des copies)."""
```

```
    __slots__ = ("_y", "_m", "_d")
```

```
    def __init__(self, year: int, month: int, day: int):
        # TODO: valider et affecter _y, _m, _d ; lever InvalidDateError si
        invalide
        pass
```

```
    @property
    def year(self) -> int: return self._y
    @property
    def month(self) -> int: return self._m
    @property
    def day(self) -> int: return self._d
```

```
    def __repr__(self) -> str:
        # TODO: format non ambigu
        return ""
```

```
    def __str__(self) -> str:
        # TODO: format lisible YYYY-MM-DD
        return ""
```

```
    def to_iso(self) -> str:
        # TODO
        return ""
```

```
    @classmethod
    def from_iso(cls, s: str) -> "Date":
        # TODO: parser, lever DateParseError si invalide
        return cls(1970, 1, 1)
```

```
    def is_leap_year(self) -> bool:
```

```

        # TODO
        return False

    def _days_in_month(self, y: int, m: int) -> int:
        # TODO
        return 30

    def day_of_year(self) -> int:
        # TODO
        return 0

    def add_days(self, n: int) -> "Date":
        # TODO: retourner une nouvelle date, gérer dépassements
        mois/année
        return self

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Date):
            return NotImplemented
        # TODO
        return False

    def __lt__(self, other: "Date") -> bool:
        # TODO: ordre lexicographique (y,m,d)
        return False

# --- Tests rapides ---
if __name__ == "__main__":
    d = Date(2024, 2, 29)
    assert d.is_leap_year() is True
    assert str(d) == "2024-02-29"
    assert Date.from_iso("2025-09-22").to_iso() == "2025-09-22"

    try:
        Date(2023, 2, 29)
        assert False, "Devrait lever InvalidDateError"
    except InvalidDateError:
        pass

    try:
        Date.from_iso("2025-13-01")
        assert False, "Devrait lever DateParseError"
    except DateParseError:
        pass

```

```
# add_days et ordre
d1 = Date(2025, 12, 31).add_days(1)
assert str(d1) == "2026-01-01"
assert Date(2025, 1, 1) < Date(2025, 1, 2)

print("Tests Ex1 OK")
```

Code corrige

```
from __future__ import annotations
from functools import total_ordering
from typing import Tuple
from errors import InvalidDateError, DateParseError

@total_ordering
class Date:
    """Date grégorienne minimale, immuable en pratique (méthodes
    renvoient des copies)."""

    __slots__ = ("_y", "_m", "_d")

    def __init__(self, year: int, month: int, day: int):
        if not (1 <= year <= 9999):
            raise InvalidDateError(f"Année invalide : {year}")
        if not (1 <= month <= 12):
            raise InvalidDateError(f"Mois invalide : {month}")

        dim = self._days_in_month(year, month)
        if not (1 <= day <= dim):
            raise InvalidDateError(f"Jour invalide : {day} pour
            {month}/{year}")

        self._y, self._m, self._d = year, month, day

    @property
    def year(self) -> int:
        return self._y

    @property
    def month(self) -> int:
        return self._m

    @property
    def day(self) -> int:
        return self._d
```

```
def __repr__(self) -> str:  
    return f"Date({self._y},{self._m:02d},{self._d:02d})"
```

```
def __str__(self) -> str:  
    return f"{self._y:04d}-{self._m:02d}-{self._d:02d}"
```

```
def to_iso(self) -> str:  
    return str(self)
```

```
@classmethod  
def from_iso(cls, s: str) -> "Date":  
    try:  
        parts = s.split("-")  
        if len(parts) != 3:  
            raise ValueError  
        y, m, d = map(int, parts)  
        return cls(y, m, d)  
    except ValueError:  
        raise DateParseError(f"Format de date invalide : {s}")
```

```
def is_leap_year(self) -> bool:  
    y = self._y  
    return (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)
```

```
def _days_in_month(self, y: int, m: int) -> int:  
    if m in (1, 3, 5, 7, 8, 10, 12):  
        return 31  
    elif m in (4, 6, 9, 11):  
        return 30  
    elif m == 2:  
        return 29 if ((y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)) else 28  
    else:  
        raise InvalidDateError(f"Mois invalide : {m}")
```

```
def day_of_year(self) -> int:  
    days = self._d  
    for m in range(1, self._m):  
        days += self._days_in_month(self._y, m)  
    return days
```

```
def add_days(self, n: int) -> "Date":  
    y, m, d = self._y, self._m, self._d  
    d += n  
    while True:
```

```

        dim = self._days_in_month(y, m)
        if d > dim:
            d -= dim
            m += 1
            if m > 12:
                m = 1
                y += 1
            if y > 9999:
                raise InvalidDateError("Dépassement d'année maximale")
        elif d < 1:
            m -= 1
            if m < 1:
                m = 12
                y -= 1
            if y < 1:
                raise InvalidDateError("Dépassement d'année minimale")
        d += self._days_in_month(y, m)
    else:
        break
    return Date(y, m, d)

```

```

def __eq__(self, other: object) -> bool:
    if not isinstance(other, Date):
        return NotImplemented
    return (self._y, self._m, self._d) == (other._y, other._m, other._d)

```

```

def __lt__(self, other: "Date") -> bool:
    if not isinstance(other, Date):
        return NotImplemented
    return (self._y, self._m, self._d) < (other._y, other._m, other._d)

```

```

# --- Tests rapides ---
if __name__ == "__main__":
    d = Date(2024, 2, 29)
    assert d.is_leap_year() is True
    assert str(d) == "2024-02-29"
    assert Date.from_iso("2025-09-22").to_iso() == "2025-09-22"

```

```

try:
    Date(2023, 2, 29)
    assert False, "Devrait lever InvalidDateError"
except InvalidDateError:
    pass

```

```

try:
    Date.from_iso("2025-13-01")
    assert False, "Devrait lever DateParseError"
except DateParseError:
    pass

d1 = Date(2025, 12, 31).add_days(1)
assert str(d1) == "2026-01-01"
assert Date(2025, 1, 1) < Date(2025, 1, 2)

print("✓ Tests Ex1 OK")

```

Exercice 2 — DateRange & recouvrement (30 min)

Objectifs : types composés, invariants, égalité, méthodes métier + erreurs.

Énoncé :

Créez `daterange.py` avec une classe `DateRange(start:Date, end:Date)` incluse (les bornes inclusives).

- Lever `InvalidDateError` si `end < start`.
- Méthodes :
 - `duration(self) -> int` (nb de jours, ex. `[2025-09-01..2025-09-01] -> 1`)
 - `contains(self, d:Date) -> bool`
 - `overlaps(self, other:"DateRange") -> bool`
 - `intersection(self, other:"DateRange") -> "DateRange | None"`

Code de départ (`daterange.py`) :

```

from __future__ import annotations
from datex import Date
from errors import InvalidDateError

```

```

class DateRange:
    __slots__ = ("start", "end")

```

```

    def __init__(self, start: Date, end: Date):

```

```

    # TODO: valider start <= end
    pass

    def duration(self) -> int:
        # TODO
        return 0

    def contains(self, d: Date) -> bool:
        # TODO
        return False

    def overlaps(self, other: "DateRange") -> bool:
        # TODO
        return False

    def intersection(self, other: "DateRange") -> "DateRange | None":
        # TODO
        return None

# --- Tests rapides ---
if __name__ == "__main__":
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
    b = DateRange(Date(2025, 9, 5), Date(2025, 9, 15))
    c = a.intersection(b)
    assert c is not None and c.start.to_iso() == "2025-09-05" and
c.end.to_iso() == "2025-09-10"
    assert a.duration() == 10
    assert a.contains(Date(2025, 9, 1)) and a.contains(Date(2025, 9, 10))
    print("Tests Ex2 OK")

```

Code corrige

```

from __future__ import annotations
from datex import Date
from errors import InvalidDateError

class DateRange:
    __slots__ = ("start", "end")

    def __init__(self, start: Date, end: Date):
        if end < start:
            raise InvalidDateError("La date de fin ne peut pas être
antérieure à la date de début.")
        self.start = start

```



```

    self.end = end

def duration(self) -> int:
    """Renvoie le nombre de jours inclusifs dans l'intervalle."""
    # Calcul basé sur les différences en jours depuis une date de
    référence.
    days = 0
    current = self.start
    while current < self.end:
        current = current.add_days(1)
        days += 1
    # intervalle inclusif → ajouter 1 jour pour inclure start
    return days + 1

def contains(self, d: Date) -> bool:
    """Vérifie si une date est incluse dans l'intervalle (bornes
    inclusives)."""
    return self.start <= d <= self.end

def overlaps(self, other: "DateRange") -> bool:
    """Renvoie True si les deux intervalles se recouvrent."""
    return not (self.end < other.start or other.end < self.start)

def intersection(self, other: "DateRange") -> "DateRange | None":
    """Renvoie la plage d'intersection ou None s'il n'y a pas de
    recouvrement."""
    if not self.overlaps(other):
        return None
    start = max(self.start, other.start)
    end = min(self.end, other.end)
    return DateRange(start, end)

def __repr__(self) -> str:
    return f"DateRange({self.start!r}, {self.end!r})"

# --- Tests rapides ---
if __name__ == "__main__":
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
    b = DateRange(Date(2025, 9, 5), Date(2025, 9, 15))
    c = a.intersection(b)
    assert c is not None and c.start.to_iso() == "2025-09-05" and
    c.end.to_iso() == "2025-09-10"
    assert a.duration() == 10
    assert a.contains(Date(2025, 9, 1)) and a.contains(Date(2025, 9, 10))

```

```
print("✓ Tests Ex2 OK")
```

Partie 2 : Sérialisation Texte vs Binaire (45 minutes)

Exercice 3 — JSON/CSV (texte) & Pickle (binaire) (45 min)

Objectifs : sérialiser une collection d'objets métier, comparer les formats.

Énoncé :

Créez `serialization.py` avec des fonctions pour sérialiser une liste de `Date` :

- Texte JSON (JSON Lines) : `dump_dates_jsonl(dates, path)` , `load_dates_jsonl(path)`
Une ligne par objet : `{"year":2025,"month":9,"day":22}`
- CSV : `dump_dates_csv(dates, path)` , `load_dates_csv(path)`
- Binaire (pickle) : `dump_dates_pickle(dates, path)` , `load_dates_pickle(path)`

Contraintes :

- Validation lors du chargement : lever `DateParseError` si enregistrement invalide.
- Gérer proprement I/O : `with open(..., "w", encoding="utf-8")` .
- Fournir un mini-bench : sauvegarder 10000 dates, comparer tailles de fichiers.

Code de départ (`serialization.py`) :

```
from __future__ import annotations
import json, csv, pickle, os
from typing import Iterable, List
from datex import Date
from errors import DateParseError
```

```
def dump_dates_jsonl(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass
```

```
def load_dates_jsonl(path: str) -> List[Date]:
    # TODO
    return []
```

```
def dump_dates_csv(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass
```

```

def load_dates_csv(path: str) -> List[Date]:
    # TODO
    return []

def dump_dates_pickle(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass

def load_dates_pickle(path: str) -> List[Date]:
    # TODO
    return []

# --- Tests/bench ---
if __name__ == "__main__":
    # Générer des dates
    base = Date(2025, 1, 1)
    dates = [base.add_days(i) for i in range(10000)]

    dump_dates_jsonl(dates, "dates.jsonl")
    dump_dates_csv(dates, "dates.csv")
    dump_dates_pickle(dates, "dates.pkl")

    lj = load_dates_jsonl("dates.jsonl")
    lc = load_dates_csv("dates.csv")
    lp = load_dates_pickle("dates.pkl")

    assert lj[:3][0].to_iso() == "2025-01-01"
    assert lc[-1].to_iso() == "2052-05-18" # 10000e jour
    assert lp[1234].to_iso() == dates[1234].to_iso()

    sizes = {p: os.path.getsize(p) for p in
["dates.jsonl", "dates.csv", "dates.pkl"]}
    print("Tailles (octets):", sizes)
    # Discussion attendue en compte-rendu: lisibilité vs compacité vs
portabilité

```

Code corrige

```

from __future__ import annotations
import json, csv, pickle, os
from typing import Iterable, List
from datex import Date
from errors import DateParseError

```

```
# --- JSON Lines ---
def dump_dates_jsonl(dates: Iterable[Date], path: str) -> None:
    """Sauvegarde une liste de Date en format JSON Lines (une ligne par
    objet)."""
    with open(path, "w", encoding="utf-8") as f:
        for d in dates:
            record = {"year": d.year, "month": d.month, "day": d.day}
            f.write(json.dumps(record) + "\n")
```

```
def load_dates_jsonl(path: str) -> List[Date]:
    """Charge une liste de Date depuis un fichier JSON Lines."""
    dates = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            try:
                record = json.loads(line)
                y, m, d = record["year"], record["month"], record["day"]
                dates.append(Date(y, m, d))
            except Exception as e:
                raise DateParseError(f"Erreur JSONL : {e}")
    return dates
```

```
# --- CSV ---
def dump_dates_csv(dates: Iterable[Date], path: str) -> None:
    """Sauvegarde les dates dans un fichier CSV avec colonnes year,
    month, day."""
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["year", "month", "day"])
        for d in dates:
            writer.writerow([d.year, d.month, d.day])
```

```
def load_dates_csv(path: str) -> List[Date]:
    """Charge une liste de Date depuis un fichier CSV."""
    dates = []
    with open(path, "r", newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                y, m, d = int(row["year"]), int(row["month"]), int(row["day"])
                dates.append(Date(y, m, d))
```

```

        except Exception as e:
            raise DateParseError(f"Erreur CSV : {e}")
    return dates

# --- Pickle (binaire) ---
def dump_dates_pickle(dates: Iterable[Date], path: str) -> None:
    """Sauvegarde les dates en format binaire (pickle)."""
    with open(path, "wb") as f:
        pickle.dump(list(dates), f)

def load_dates_pickle(path: str) -> List[Date]:
    """Charge les dates en format pickle."""
    try:
        with open(path, "rb") as f:
            obj = pickle.load(f)
            if not all(isinstance(d, Date) for d in obj):
                raise DateParseError("Contenu Pickle invalide.")
            return obj
    except Exception as e:
        raise DateParseError(f"Erreur Pickle : {e}")

# --- Tests/bench ---
if __name__ == "__main__":
    # Générer des dates
    base = Date(2025, 1, 1)
    dates = [base.add_days(i) for i in range(10000)]

    dump_dates_jsonl(dates, "dates.jsonl")
    dump_dates_csv(dates, "dates.csv")
    dump_dates_pickle(dates, "dates.pkl")

    lj = load_dates_jsonl("dates.jsonl")
    lc = load_dates_csv("dates.csv")
    lp = load_dates_pickle("dates.pkl")

    assert lj[0].to_iso() == "2025-01-01"
    assert lc[-1].to_iso() == "2052-05-18" # 10000e jour
    assert lp[1234].to_iso() == dates[1234].to_iso()

    sizes = {p: os.path.getsize(p) for p in ["dates.jsonl", "dates.csv",
"dates.pkl"]}
    print("✓ Tests Ex3 OK")

```

```
print("Tailles (octets):", sizes)
```

Partie 3 : Gestion des erreurs / exceptions (30 minutes)

Exercice 4 — API robuste & journalisation légère (30 min)

Objectifs : maîtriser `try / except / else / finally`, exceptions métiers, messages utiles.

Énoncé :

Créez `repo.py` : un dépôt simple pour stocker des `DateRange` en `JSONL`.

- Exceptions :

```
class RepositoryError(Exception): ...
class RepositoryIOError(RepositoryError): ...
class RepositoryDataError(RepositoryError): ...
```

- Classe `DateRangeRepository(path:str)` avec :
 - `save(self, ranges:list[DateRange]) -> None`
 - `load(self) -> list[DateRange]`
- Règles :
 - I/O : capturer `OSError` → `RepositoryIOError` (enrichir le message avec le chemin).
 - Données invalides (bornes inversées, champs manquants) → `RepositoryDataError`.
 - Utiliser `try / except / else / finally` pour séparer logique et nettoyage.
 - (Optionnel) journalisation minimaliste avec `print("[INFO] ...")` pour l'exercice.

Code de départ (repo.py) :

```
from __future__ import annotations
import json
from typing import List, Dict, Any
from datex import Date
from daterange import DateRange
from errors import InvalidDateError
from serialization import DateParseError # ou depuis errors si déplacé
```

```
class RepositoryError(Exception): ...
class RepositoryIOError(RepositoryError): ...
class RepositoryDataError(RepositoryError): ...
```

```
class DateRangeRepository:
```

```

def __init__(self, path: str):
    self._path = path

def save(self, ranges: List[DateRange]) -> None:
    # TODO: sérialiser en JSONL avec schéma
    {"start": "YYYY-MM-DD", "end": "YYYY-MM-DD"}
    pass

def load(self) -> List[DateRange]:
    # TODO: I/O + validation + mapping vers objets
    return []

# --- Tests rapides ---
if __name__ == "__main__":
    repo = DateRangeRepository("ranges.jsonl")
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
    b = DateRange(Date(2025, 9, 20), Date(2025, 9, 21))
    repo.save([a, b])
    loaded = repo.load()
    assert len(loaded) == 2 and loaded[0].start.to_iso() == "2025-09-01"
    print("Tests Ex4 OK")

```

Code corrige pour [repo.py](#)

```

from __future__ import annotations
import json
from typing import List
from datex import Date
from daterange import DateRange
from errors import InvalidDateError, DateParseError

```

```

# --- Exceptions spécifiques ---
class RepositoryError(Exception):
    """Erreur générale du dépôt."""
    pass

```

```

class RepositoryIOError(RepositoryError):
    """Erreur d'entrée/sortie du dépôt."""
    pass

```

```

class RepositoryDataError(RepositoryError):
    """Erreur de données dans le dépôt."""
    pass

```

```

# --- Classe principale ---

```

```

class DateRangeRepository:
    """Gestionnaire de stockage des objets DateRange en JSONL."""
    def __init__(self, path: str):
        self._path = path

    def save(self, ranges: List[DateRange]) -> None:
        """Sauvegarde une liste de DateRange en JSON Lines."""
        print(f"[INFO] Sauvegarde de {len(ranges)} intervalles dans {self._path} ...")
        try:
            with open(self._path, "w", encoding="utf-8") as f:
                for r in ranges:
                    record = {"start": r.start.to_iso(), "end": r.end.to_iso()}
                    json.dump(record, f)
                    f.write("\n")
        except OSError as e:
            raise RepositoryIOError(f"Erreur d'écriture du fichier {self._path}: {e}")

    def load(self) -> List[DateRange]:
        """Charge les DateRange depuis le fichier JSONL."""
        print(f"[INFO] Chargement depuis {self._path} ...")
        ranges: List[DateRange] = []
        try:
            with open(self._path, "r", encoding="utf-8") as f:
                for line in f:
                    try:
                        record = json.loads(line)
                        if "start" not in record or "end" not in record:
                            raise RepositoryDataError(f"Champs manquants : {record}")
                        start = Date.from_iso(record["start"])
                        end = Date.from_iso(record["end"])
                        if end < start:
                            raise RepositoryDataError(f"Borne inversée : {record}")
                        ranges.append(DateRange(start, end))
                    except (DateParseError, InvalidDateError, RepositoryDataError) as e:
                        raise RepositoryDataError(f"Erreur de données : {e}")
        except OSError as e:
            raise RepositoryIOError(f"Erreur de lecture du fichier {self._path}: {e}")
        finally:
            print(f"[INFO] Chargement terminé ({len(ranges)} intervalles valides).")

```



```
return ranges
```

```
# --- Tests rapides ---
```

```
if __name__ == "__main__":
```

```
    repo = DateRangeRepository("ranges.jsonl")
```

```
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
```

```
    b = DateRange(Date(2025, 9, 20), Date(2025, 9, 21))
```

```
    repo.save([a, b])
```

```
    loaded = repo.load()
```

```
    assert len(loaded) == 2
```

```
    assert loaded[0].start.to_iso() == "2025-09-01"
```

```
    assert loaded[1].end.to_iso() == "2025-09-21"
```

```
    print("✅ Tests Ex4 OK")
```

Partie 4 : Mini-Projet de Synthèse (45 minutes)

Exercice 5 — Gestion d'« Événements » avec dépôt pluggable

Objectifs : intégrer `Date`, `DateTime`, sérialisation, exceptions, et interface dépôt.

Contexte :

On veut gérer des événements avec une date unique ou une période. Chaque événement a : `title:str`, `period:DateTime`, `tags:list[str]`.

Énoncé :

Créez `events.py` et `event_repo.py`.

1. Event :

- `__init__(self, title:str, period:DateTime, tags:list[str] | None=None)`
- `__repr__`, `__str__` → ex: `Event('Conf IAG', [2025-10-06..2025-10-06], tags=['univ', 'iag'])`
- `overlaps(self, other: "Event") -> bool` (via `DateTime.overlaps`)

2. Dépôts :

- Interface minimale `BaseEventRepository` (classe abstraite simple) avec `save(events)`, `load()`.
- Implémentations :
 - ◁ `EventJsonRepository(path)` : texte `JSONL` (lisible).
 - ◁ `EventBinaryRepository(path)` : binaire (pickle).

3. Script `main.py` :

- Créer 5-10 événements (mélanger dates uniques et périodes).
- Sauvegarder via `JSONL` puis `Pickle`.
- Recharger et vérifier :
 - ◁ même nombre,
 - ◁ détection de conflits (deux événements qui se chevauchent).
- Afficher la taille des fichiers et un court commentaire (« texte plus portable/lisible, binaire plus compact/rapide mais moins sûr et moins interopérable »).

Squelettes :

Code non corrige

```
# events.py
```

```
from __future__ import annotations
```

```
from typing import List
```

```
from datetime import DateTime
```

```
class Event:
```

```
    __slots__ = ("title", "period", "tags")
```

```
    def __init__(self, title: str, period: DateTime, tags: List[str] | None = None):
```

```
        # TODO: valider title non vide, copier tags or []
```

```
        pass
```

```
    def overlaps(self, other: "Event") -> bool:
```

```
        # TODO
```

```
        return False
```

```
    def __repr__(self) -> str:
```

```
# TODO
return ""
def __str__(self) -> str:
# TODO
return ""
```

```
# event_repo.py
from __future__ import annotations
import json, pickle, os
from typing import List, Dict, Any
from events import Event
from datex import Date
from daterange import DateRange
from repo import RepositoryError, RepositoryIOError,
RepositoryDataError
```

```
class BaseEventRepository:
    def save(self, events: List[Event]) -> None: raise NotImplementedError
    def load(self) -> List[Event]: raise NotImplementedError
```

```
class EventJsonlRepository(BaseEventRepository):
    def __init__(self, path: str): self._path = path
    def save(self, events: List[Event]) -> None:
        # TODO: {"title":..., "start":"YYYY-MM-DD", "end":"YYYY-MM-DD",
"tags":[...]}
        pass
    def load(self) -> List[Event]:
        # TODO: validations + exceptions
        return []
```

```
class EventBinaryRepository(BaseEventRepository):
    def __init__(self, path: str): self._path = path
    def save(self, events: List[Event]) -> None:
        # TODO: pickle.dump
        pass
    def load(self) -> List[Event]:
        # TODO: pickle.load + validations minimales
        return []
```

[main.py](#)

```
# main.py (mini-projet)
from __future__ import annotations
from events import Event
from daterange import DateRange
from datex import Date
```

```

from event_repo import EventJsonRepository, EventBinaryRepository

if __name__ == "__main__":
    evs = [
        Event("Conf IAG", DateRange(Date(2025,10,6), Date(2025,10,6)),
["univ","iag"]),
        Event("Soutenance", DateRange(Date(2025,11,3), Date(2025,11,3)),
["phd"]),
        Event("Vacances Toussaint", DateRange(Date(2025,10,25),
Date(2025,11,2)), ["feries"]),
        Event("Jury", DateRange(Date(2025,11,2), Date(2025,11,2)

```

Code corrige 🌱 events.py

```

from __future__ import annotations
from typing import List
from datarange import DateRange

```

```

class Event:

```

```

    __slots__ = ("title", "period", "tags")

```

```

    def __init__(self, title: str, period: DateRange, tags: List[str] | None =
None):

```

```

        if not title or not isinstance(title, str):

```

```

            raise ValueError("Le titre d'un événement ne peut pas être vide.")

```

```

        self.title = title.strip()

```

```

        self.period = period

```

```

        self.tags = list(tags) if tags else []

```

```

    def overlaps(self, other: "Event") -> bool:

```

```

        """Retourne True si les périodes des événements se chevauchent."""

```

```

        return self.period.overlaps(other.period)

```

```

    def __repr__(self) -> str:

```

```

        return f"Event({self.title!r}, {self.period!r}, tags={self.tags!r})"

```

```

    def __str__(self) -> str:

```

```

        return f"Event('{self.title}',

```

```

[{self.period.start.to_iso()}..{self.period.end.to_iso()}], tags={self.tags})"

```

🌱 event_repo.py

```

from __future__ import annotations
import json, pickle, os
from typing import List, Dict, Any
from events import Event

```

```
from datex import Date
from daterange import DateRange
from repo import RepositoryError, RepositoryIOError,
RepositoryDataError
```

```
class BaseEventRepository:
    """Interface minimale pour les dépôts d'événements."""
    def save(self, events: List[Event]) -> None:
        raise NotImplementedError
    def load(self) -> List[Event]:
        raise NotImplementedError
```

```
# --- JSONL ---
```

```
class EventJsonRepository(BaseEventRepository):
    def __init__(self, path: str):
        self._path = path

    def save(self, events: List[Event]) -> None:
        print(f"[INFO] Sauvegarde JSONL dans {self._path} ...")
        try:
            with open(self._path, "w", encoding="utf-8") as f:
                for e in events:
                    rec = {
                        "title": e.title,
                        "start": e.period.start.to_iso(),
                        "end": e.period.end.to_iso(),
                        "tags": e.tags,
                    }
                    f.write(json.dumps(rec) + "\n")
        except OSError as e:
            raise RepositoryIOError(f"Erreur d'écriture JSONL : {e}")
```

```
    def load(self) -> List[Event]:
        print(f"[INFO] Chargement JSONL depuis {self._path} ...")
        events: List[Event] = []
        try:
            with open(self._path, "r", encoding="utf-8") as f:
                for line in f:
                    try:
                        obj = json.loads(line)
                        for k in ("title", "start", "end", "tags"):
                            if k not in obj:
                                raise RepositoryDataError(f"Champ manquant: {obj}")
```

```

        start = Date.from_iso(obj["start"])
        end = Date.from_iso(obj["end"])
        if end < start:
            raise RepositoryDataError(f"Borne inversée : {obj}")
        period = DateRange(start, end)
        events.append(Event(obj["title"], period, obj["tags"]))
    except RepositoryDataError as e:
        print(f"[WARN] Ligne ignorée : {e}")
    except OSError as e:
        raise RepositoryIOError(f"Erreur de lecture JSONL : {e}")
    print(f"[INFO] {len(events)} événements chargés.")
    return events

```

--- Pickle ---

```

class EventBinaryRepository(BaseEventRepository):

```

```

    def __init__(self, path: str):
        self._path = path

```

```

    def save(self, events: List[Event]) -> None:
        print(f"[INFO] Sauvegarde binaire (pickle) dans {self._path} ...")
        try:
            with open(self._path, "wb") as f:
                pickle.dump(events, f)
        except OSError as e:
            raise RepositoryIOError(f"Erreur d'écriture Pickle : {e}")

```

```

    def load(self) -> List[Event]:
        print(f"[INFO] Chargement binaire (pickle) depuis {self._path} ...")
        try:
            with open(self._path, "rb") as f:
                events = pickle.load(f)
            if not all(isinstance(e, Event) for e in events):
                raise RepositoryDataError("Objet non Event détecté dans le
fichier Pickle.")
            print(f"[INFO] {len(events)} événements chargés.")
            return events
        except OSError as e:
            raise RepositoryIOError(f"Erreur de lecture Pickle : {e}")
        except (pickle.UnpicklingError, RepositoryDataError) as e:
            raise RepositoryDataError(f"Erreur de données Pickle : {e}")

```

Compte rendu

COMPTE RENDU — TP Python (Dates, Sérialisation & Dépôts)

Objectifs globaux

- Créer et manipuler des classes métier ([Date](#), [DateRange](#), [Event](#)).
- Gérer les erreurs et exceptions personnalisées.
- Implémenter des méthodes de validation, parsing et comparaison.
- Sérialiser les objets avec JSONL, CSV et Pickle.
- Concevoir des dépôts (repositories) robustes et interchangeables.
- Appliquer les principes try / except / finally et la journalisation simple.